

Consolidated Dashboard “Refresh” Feature — Scope Summary

Project: DevOps HQ Monitoring Dashboard

Feature: Real-time Consolidated Report with Refresh Button

Status: Scope complete, ready for sprint planning

Prepared by: Selym (Design/Scoping Agent)

Date: 2026-04-09 19:00 EDT

What We’re Building

A “Refresh” button on DevOps HQ that generates a real-time consolidated report showing:

- **Task Status** breakdown (proposed, in-progress, completed, blocked)
- **Blockers** (what’s stuck, why, who’s responsible)
- **Owner** (who’s assigned to each task)
- **Risk Level** (high/medium/low, color-coded)

User flow: 1. User clicks [Refresh ♻️] button in top-right of HQ 2. Dashboard shows loading spinner (2-5 seconds) 3. Report appears: summary cards + critical blockers + task table 4. User can filter by status/owner, sort by any column, export to CSV/PDF/Slack

Design Decisions

1. UI Component Placement

- ☐ **Refresh button** in top-right header (next to settings icon)
- ☐ **Report display** in main dashboard area (4 zones: summary, blockers, task table, export)
- ☐ **Layout** responsive (desktop, tablet, mobile)

2. Report Generation Trigger

- **Primary:** onclick handler (user clicks button)
- **Secondary:** React Query cache (3-5 min TTL for deduping)
- **Debounce:** 500ms (prevent double-clicks)

3. Display & Preview

- **Real-time:** No “save” step; preview is the final report
- **Zones:** Summary cards → Critical blockers → Task snapshot table
- **Actions:** Inline buttons on blockers (Unblock, Escalate, Reassign)

4. Visible Fields

Per Task: - Task ID, Title, Status (badge), Owner (avatar), Risk (color), % Complete, Last Updated

Per Blocker: - Task ID, Reason, Owner, Risk Level, Age, Suggested Action

Summary: - Total tasks, % Completed, # Blocked, # At-Risk, Velocity (tasks/week)

5. Architecture: Hybrid API + CLI

- **REST API** (primary): Real-time UI updates, client-side caching with React Query
 - **CLI fallback** (secondary): Batch reports, scheduled jobs, Slack automation
 - **Caching:** Redis (server) + React Query (client) for <1s response on refresh
-

Deliverables from This Scope Session

□ Documentation (4 files created)

1. **UI_DESIGN_DOC.md** — Component design, layout, fields, mockup (11.8 KB)
2. **ARCHITECTURE_RECOMMENDATION.md** — Tech stack, data flow, performance targets (13.2 KB)
3. **IMPLEMENTATION_PLAN.md** — Day-by-day breakdown, resource allocation, risks, acceptance criteria (17 KB)
4. **WARD_BACKEND_HANDOFF.md** — Backend handoff with API contract, schema, 7 critical questions (14.3 KB)

□ Key Artifacts

- **API Contract:** POST /api/v1/dashboard/consolidated-report with full request/response schema
 - **Database Schema:** blockers table, indexes, RLS policies (SQL provided)
 - **Mockup/Wireframe:** ASCII mockup showing layout + visual hierarchy
 - **Time Estimate:** 4-5 calendar days (2 devs in parallel)
-

Timeline at a Glance

Week Starting Monday 2026-04-14:

Fri 4/11 (Day 1) | Component skeleton + database schema
Mon 4/14 (Day 2) | Core features (table, blockers, export)
Tue 4/15 (Day 3) | Integration begins (API connection)
Wed 4/16 (Day 4) | Testing + staging deployment
Thu 4/17 (Day 5) | Production release

MVP Ready: Thursday 2026-04-17 (end of week)

Effort Breakdown

Component	Owner	Hours	Days
Frontend UI	Frontend Dev	18-21	2-2.5
Backend API	Ward	14-20	2-2.5
Testing + Integration	Both	4-6	0.5-1
Total		36-47	4-5

Critical Questions Needing Ward's Answers (by EOD Fri 4/11)

These determine implementation details:

1. **Risk Level:** Auto-calculated or manual field?
2. **Blocker Detection:** Status = 'blocked', or auto-detect from blockers table?
3. **Multiple Owners:** Single owner (assigned_to) or multiple owners?
4. **Team Scope:** Single team or multi-team report?
5. **Real-Time Updates:** Use Supabase Realtime or polling?
6. **Data Volume:** How many tasks per team? (50? 500? 5000?)

7. **Caching Infrastructure:** Redis available, or new setup?

☐ **Detailed Q&A** in `WARD_BACKEND_HANDOFF.md` (sections Q1-Q7)

Success Metrics

- ☐ **Latency:** API responds in <5s (cold), <1s (cached)
 - ☐ **Functionality:** All 4 fields visible (status, blockers, owner, risk)
 - ☐ **Security:** RLS enforced, users only see team data
 - ☐ **UX:** Refresh button clickable, export works, sorting/filtering functional
 - ☐ **Quality:** All tests passing, zero critical bugs at launch
 - ☐ **Performance:** Cache 80%+ hit rate, P95 latency <8s at 100 req/s
-

Risk Summary

Risk	Probability	Impact	Mitigation
API latency >5s	Medium	Users wait too long	Daily benchmarks, optimize indexes
Data schema mismatch	Low	Frontend/backend misalignment	Sync on Day 2, share contract early
RLS policy bug	Low	Users see wrong data	Test with multiple roles in staging
Integration delay	Medium	Idle time between teams	Mock API on Day 1, parallel work
Scope creep	High	Phase 2 leaks into MVP	Lock scope, use feature flags

Architecture Overview

Data Flow (Simple)

User clicks [Refresh ↻]
↓
API call: `POST /api/v1/dashboard/consolidated-report`

↓
Backend: Aggregate tasks + blockers from Supabase
↓
Cache result in Redis (TTL 5 min)
↓
Return JSON to frontend
↓
React Query caches (TTL 3 min)
↓
Dashboard renders: summary + blockers + table

Tech Stack

Frontend: React 18 + TypeScript + React Query + TanStack Table + Tailwind

Backend: Node.js + Express + Supabase + Redis

Database: PostgreSQL (Supabase) with RLS policies

Recommended Approach

Phase 1 (MVP): 5 days

- ☐ Refresh button + report display
- ☐ Task status, blockers, owner, risk visible
- ☐ CSV export
- ☐ <5s latency, caching working

Phase 1.5 (Hardening): 1-2 days (following week)

- ☐ Add Redis caching (if not in MVP)
- ☐ PDF export
- ☐ Load testing + performance tuning
- ☐ Auto-refresh toggle (optional)

Phase 2 (Enhancement): Future sprint

- ☐ WebSocket real-time updates
 - ☐ Slack integration
 - ☐ Scheduled reports (cron)
 - ☐ Historical trends/analytics
-

Files in Workspace

All design documents are committed to:
/data/.openclaw/workspace/projects/myxxit-devops/

Files: - UI_DESIGN_DOC.md (design + mockup) -
ARCHITECTURE_RECOMMENDATION.md (tech stack + performance) -
IMPLEMENTATION_PLAN.md (timeline + breakdown) -
WARD_BACKEND_HANDOFF.md (backend handoff + questions) -
CONSOLIDATED_DASHBOARD_SCOPE_SUMMARY.md (this file)

Next Steps

1. **Approval:** Travis reviews & approves design direction
 2. **Questions:** Ward answers critical Q&A (by EOD Fri 4/11)
 3. **Kickoff:** Sprint planning meeting Friday 2026-04-11
 4. **Dev:** Parallel frontend + backend work (Fri-Thu)
 5. **Release:** Production deployment Thursday 2026-04-17
-

Approvals Needed

☐

Travis: Approve design + scope

☐

Ward: Approve API contract + answer critical questions

☐

Frontend Lead: Approve timeline + resource allocation

☐

DevOps: Approve deployment plan + infrastructure needs

Summary

What: Real-time consolidated report with refresh button on DevOps HQ

When: Ready for production by 2026-04-17 (5 days from kickoff)

Who: Frontend Dev + Ward (Backend) + QA

Effort: ~40-50 hours total (2 developers in parallel)

Risk: Medium (scope creep, data mismatch) — mitigated by clear contract + sync meetings

Status: ☐ Scope complete, ready for sprint planning

Questions? Reach out. All design decisions documented and ready for review.